



Sass. Part 2.

SCSS



Mixins. Content

У миксинов есть возможность принимать не только аргументы, но и участки стилей, которые мы можем передать в момент, когда делаем `@include` миксина. Это позволяет создавать гибкие реиспользуемые конструкции. Делается это с помощью директивы `@content`. Например:

```
@mixin minMedia($width) {  
  
  @media screen and (min-width: $width) {  
  
    @content;  
  
  }  
  
}  
  
@include minMedia(600px) {  
  
  display: flex;  
  
}
```



Arglist

Бывают случаи, когда мы заранее не знаем точное количество аргументов, которые может принимать миксин. Тогда в аргументы мы можем указать переменную типа `arglist`:

```
@mixin linear-gradient($direction, $gradients...) {  
  
  background-color: nth($gradients, 1);  
  
  background-image: linear-gradient($direction, $gradients...);  
  
}
```

Этот миксин может принимать любое количество аргументов, не привязываясь к их количеству



Mixin arguments

Помимо обычных переменных и `arglist` миксин может принимать как аргументы `list` и `map`.

```
$some-data: (red, bold, 5px);
```

```
@mixin doSomething($a, $b, $c: default) {
```

```
  color: $a;
```

```
  font-weight: $b;
```

```
  border-radius: $c;
```

```
}
```

```
@include doSomething($some-data...);
```



Mixin arguments

Когда таким образом мы применяем лист как набор параметров, важно соблюдать нужный нам порядок, т.к. значения листа будут подставляться в миксин по порядку.

Если же мы не хотим заботиться о порядке значений в листе, можно точно так же использовать map. Вместо порядка в map, переменные будут сопоставляться с аргументами по ключам значений в map:

```
$some-map-data: ('a': red, 'b': bold, 'c': 5px);
```

```
@include doSomething($some-map-data...);
```



Loops

Циклы - возможность перебирать листы и массивы и последовательно совершать действия над каждым элементом листа\массива (эта возможность называется - итерация)

В Sass есть три вида циклов: `@each`, `@for`, `@while`.

На практике циклы используются не так часто, но из всех больше всего используется `@each`.

Пример: `@each $key, $value in $map {`

```
.page-#{ $key } {  
  
  background: $value;  
  
}  
}
```



Conditional operators

Иногда есть потребность применять те или иные стили в зависимости от каких-то условий.

Для этого в Sass есть условные операторы: @if, @else if, @else.

Эти директивы принимают выражения, и стили внутри условия будут применяться только если выражение возвращает true. Например:

```
.container {  
  
  @if 1 < 2 {  
  
    color: red;  
  
  }
```



Media queries

Еще одной особенностью Sass есть то, что в медиа-запросы мы можем вкладывать точно в любой селектор:

```
.header {  
  
  display: inline-block;  
  
  @media screen and (min-width: 600px) {  
  
    display: flex;  
  
  }  
  
}
```




Put it all together

Пример использования map, mixin, interpolation, content, conditional вместе.

Сначала создадим переменную, где сохраним желаемые контрольные точки:

```
$breakpoints: (  
  'md': (min-width: 800px),  
  'lg': (min-width: 1000px),  
  'xl': (min-width: 1200px),  
);
```



Put it all together

```
@mixin minMedia($target) {  
  
  $breakpoint: map-get($breakpoints, $target);  
  
  @if $breakpoint {  
  
    // inspect - Returns the string representation of a value as it would be represented in Sass.  
  
    $query: inspect($breakpoint);  
  
    @media #{$query} {  
  
      @content;  
  
    }  
  
  }  
  
}
```



Imports

В Sass есть возможность разделить стили на несколько файлов без ущерба для производительности (в отличие от CSS-правила `@import`, так как с помощью последнего браузер отдельные файлы загружаются отдельно, а не параллельно). Благодаря `@import` в Sass, можно использовать множество файлов. В итоге, все они потом будут собраны в один файл `css`.



Partials

В связи с тем, что в Sass рекомендуется разделять логически связанные части в отдельные файлы, важно вынести переменные, миксины и т.п. в отдельные файлы и импортировать куда нужно.

Для того, чтобы препроцессор не создавал отдельные файлы для таких вещей, можно сказать ему об этом: Называть файлы с таким содержимым стоит начиная с символа “_”:

`_colors.scss, _mixins.scss.`

Это говорить препроцессору, что это так называемые `partials`, и их отдельно не нужно преобразовывать и создавать для них отдельный файл.



Architecture

Существует много подходов к организации файлов и папок стилей.

Один из популярных: <https://sass-guidelin.es/ru/#section-36>